

EthSwap

A decentralized token exchange built on Ethereum, enabling trustless peer-to-contract swaps between ETH and ERC-20 tokens via smart contracts.

PROJECT TYPE

Decentralized Application
(DApp)

BLOCKCHAIN

Ethereum (Solidity
0.5.x)

FRONTEND

React.js +
Web3.js

VERSION

1.0.0

Project Overview

EthSwap is a fully decentralized token exchange protocol deployed on the Ethereum blockchain. It allows users to seamlessly buy and sell **DAPP Tokens** (a custom ERC-20 token) using ETH at a fixed exchange rate, without requiring any central authority, order book, or intermediary.

The project demonstrates core DeFi concepts including smart contract deployment, ERC-20 token standards, token approvals, and decentralized liquidity pools — all connected to a modern React.js frontend via Web3.js and MetaMask.

Exchange Rate: 1 ETH = 100 DAPP Tokens (fixed rate set in the smart contract). The contract acts as the liquidity provider, holding both ETH and DAPP reserves.

Core Features



Token Swap

Buy DAPP tokens with ETH or sell DAPP tokens back for ETH instantly at a fixed rate.



Non-Custodial

All trades are executed directly on-chain. No user funds are ever held by a third party.



Event Logging

Every purchase and sale emits on-chain events (TokensPurchased, TokensSold) for full auditability.



MetaMask Integration

Connects seamlessly with MetaMask for transaction signing and account management.



Live Balance Updates

ETH and DAPP balances refresh automatically after every confirmed transaction.



Safety Checks

Smart contract enforces liquidity checks — users cannot buy or sell beyond available reserves.

Technology Stack

Layer	Technology	Version	Purpose
Smart Contracts	Solidity	^0.5.0	On-chain business logic for token swaps
Development Framework	Truffle Suite	v5.11.5	Contract compilation, migration, testing
Local Blockchain	Ganache	Latest	Local Ethereum network for development
Frontend Framework	React.js	^16.x	User interface and component management
Blockchain Library	Web3.js	^1.x	Ethereum node communication from browser
Wallet	MetaMask	Browser Extension	Transaction signing and account access
Testing	Mocha + Chai	Latest	Smart contract unit testing
Token Standard	ERC-20	Custom Implementation	DAPP Token fungible token contract

Smart Contract Architecture

The project is composed of three Solidity contracts, each with a distinct responsibility:

Token.sol — ERC-20 Token

A custom ERC-20 compliant token contract. It mints **1,000,000 DAPP tokens** on deployment and supports transfer, approve, and transferFrom operations required for the exchange flow.

```
// Key properties
name:      "DApp Token"
symbol:    "DAPP"
totalSupply: 1,000,000 × 1018 (18 decimals)
decimals:  18
```

EthSwap.sol — Exchange Contract

The core exchange contract. It holds the DAPP token reserve and ETH liquidity, and exposes two primary functions:

buyTokens() — payable

User sends ETH → contract calculates token amount ($\text{msg.value} \times \text{rate}$) → checks reserve → transfers DAPP to user → emits TokensPurchased event.

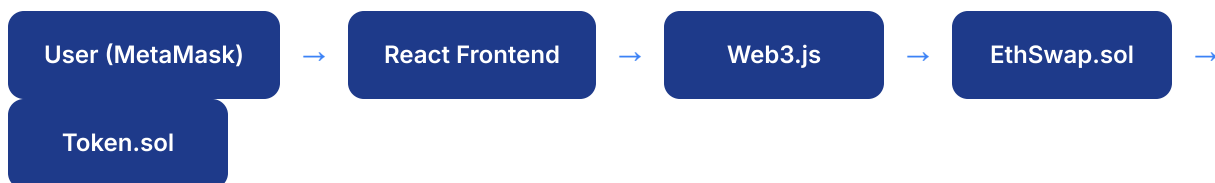
sellTokens(uint _amount)

User approves contract → calls sellTokens → contract calls transferFrom → calculates ETH ($\text{amount} \div \text{rate}$) → sends ETH to user → emits TokensSold.

```
contract EthSwap {
    Token    public token;
    uint     public rate = 100;    // 1 ETH = 100 DAPP

    function buyTokens()    public payable { ... }
    function sellTokens(uint _amount) public { ... }
    function() external payable {}    // accept ETH
}
```

Transaction Flow



Project Structure

```
eth_swap/
├── src/
│   ├── contracts/
│   │   ├── Token.sol           # ERC-20 DAPP Token
│   │   ├── EthSwap.sol        # Exchange logic
│   │   └── Migrations.sol      # Truffle migration contract
│   ├── abis/                  # Compiled contract ABIs (auto-generated)
│   └── components/
│       ├── App.js             # Root component, Web3 + state management
│       ├── Main.js            # Swap UI (buy/sell interface)
│       └── NavBar.js          # Top navigation with account + Identicon
├── migrations/
│   ├── 1_initial_migration.js
│   └── 2_deploy_contracts.js  # Deploys Token + EthSwap, seeds liquidity
├── test/
│   └── EthSwap.test.js        # Full test suite (Mocha + Chai)
└── truffle-config.js          # Network + compiler config
```

SECTION 05

Setup & Installation

Prerequisites: Node.js v14+, npm, Truffle CLI, Ganache (desktop or CLI), MetaMask browser extension.

1 Clone the repository

```
git clone https://github.com/YOUR_USERNAME/eth_swap.git
cd eth_swap
```

2 Install dependencies

```
npm install
```

3 Start Ganache

Open Ganache desktop app. Create a new workspace on port **7545**, Network ID **5777**.

4

Deploy contracts

```
truffle migrate --reset
```

5

Run tests

```
truffle test
```

6

Start the frontend

```
npm start
```

Open **http://localhost:3000** in your browser. Connect MetaMask to Localhost 7545 and import a Ganache account.

SECTION 06

Test Suite

The project includes a comprehensive test suite written with Mocha and Chai covering all core contract functionality:

Test Group	Test Case	Status
EthSwap Deployment	Contract has correct name	✓ Pass
Token Deployment	Token has correct name	✓ Pass
Token Deployment	EthSwap receives full token supply	✓ Pass
buyToken()	Investor balance increases correctly after purchase	✓ Pass
buyToken()	EthSwap token reserve decreases after purchase	✓ Pass
buyToken()	TokensPurchased event emitted with correct args	✓ Pass
sellToken()	Investor DAPP balance goes to 0 after full sell	✓ Pass
sellToken()	EthSwap token reserve restored after sell	✓ Pass

Test Group	Test Case	Status
sellToken()	TokensSold event emitted with correct args	✓ Pass
sellToken()	Selling more than balance is rejected	✓ Pass

SECTION 07

Key Concepts Demonstrated

ERC-20 Token Standard

The DAPP token implements the full ERC-20 interface including `balanceOf`, `transfer`, `approve`, `allowance`, and `transferFrom`. The `approve` → `transferFrom` pattern is critical for the sell flow.

Liquidity Pool Pattern

EthSwap acts as a primitive automated market maker (AMM). It holds reserves of both ETH and DAPP tokens and uses a fixed rate instead of a dynamic pricing curve (unlike Uniswap's $x \times y = k$ model). This makes it an ideal learning project before studying more complex AMM designs.

Web3 + React Integration

The frontend uses `window.web3` injected by MetaMask. Contract instances are created from ABI + address pairs stored in the compiled JSON artifacts under `src/abis/`. State management is handled with React class component `setState` and a polling interval for real-time balance updates.

Note on the sell flow: Selling tokens requires two separate MetaMask transactions — first `approve()` to grant EthSwap permission to spend your tokens, then `sellTokens()` to execute the actual swap. This is a standard ERC-20 security pattern.

SECTION 08

Dependencies

npm Packages

[react](#)[react-dom](#)[web3](#)[truffle](#)[chai](#)[chai-as-promised](#)[babel-register](#)[babel-polyfill](#)[identicon.js](#)[bootstrap](#)

Global Tools Required

[truffle CLI](#)[ganache \(desktop\)](#)[node v14+](#)[npm](#)[MetaMask extension](#)